

Characterizing EAGLE-3 Speculative Decoding in vLLM - Performance across cloud GPUs and Apple Silicon under high concurrency

Shreya Banga Aaryaman Bajaj Himanshu Jhawar Rajvardhan Patil

Applied Machine Learning in the Cloud (AMLC)

May 8, 2026

Abstract

Speculative decoding accelerates large language model (LLM) inference by using a lightweight draft model to propose candidate tokens that are verified in a single batched forward pass by the target model. Despite growing adoption, rigorous cross-hardware, cross-task, and high-concurrency evaluations remain scarce. We benchmark **EAGLE-3** speculative decoding against autoregressive baselines on three GPU tiers NVIDIA A100 (80 GB), NVIDIA L4 (24 GB), and Apple Silicon (Mac via MLX) across Chat, Code, and Summarization tasks at seven concurrency levels. Using **Meta-Llama-3.1-8B-Instruct** as the target model, we find that EAGLE-3 delivers $1.05\times$ – $2.91\times$ throughput gains on server GPUs with no crossover point observed up to $c = 128$ concurrent users. Our evaluation, spanning chat, code generation, and summarization tasks under realistic multi-user inference workloads, shows that speculative decoding consistently improves throughput over autoregressive decoding while also reducing infrastructure costs for LLM serving.

1. Introduction

Large language models have become central infrastructure for modern AI applications, but their sequential autoregressive generation creates a fundamental throughput bottleneck. Each generated token requires a full forward pass through the model, which for an 8B-parameter model on an NVIDIA A100 costs roughly 10–13 ms per token under typical load. At scale, this translates directly to infrastructure cost and user-facing latency.

Speculative decoding [1, 2] proposes to amortize this cost by having a small, cheap *draft model* generate γ candidate tokens ahead of time, which the large *target model* then verifies in a single forward pass over the extended sequence. Accepted draft tokens are kept; the first rejected token triggers a corrected sample from the target distribution. The key property is *losslessness*: the output distribution is provably identical to standard autoregressive generation.

While speculative decoding is conceptually elegant, its practical benefits depend heavily on (a) the draft model’s *acceptance rate* α for a given task, (b) the hardware’s ability to batch requests through the draft-verify pipeline, and (c) the concurrency level of the serving system. Prior work has not systematically explored the regime of $c > 32$ concurrent users, leaving open the question of whether speedups degrade monotonically or exhibit non-linear behavior at high load. This project fills that gap with a sweep up to $c = 128$.

2. Course Relevance

This project intersects all three core AMLC course topics.

Cloud Computing. The A100 experiments were conducted on Google Colab Pro+ and the L4 experiments on a GCP `g2-standard-4` instance. The benchmark measures real cloud cost-per-inference across the full concurrency sweep, demonstrating 37%–40% cost reductions for identical workloads.

Deep Neural Networks. The target model (**Meta-Llama-3.1-8B-Instruct**) is a transformer-based LLM. EAGLE-3 is a learned draft head a lightweight neural network attached to the target model’s hidden states that learns to predict the target’s next-token distribution. Analyzing how draft-head acceptance rates vary across tasks and concurrency levels is central to this work.

Performance Analysis. The benchmark measures Time to First Token (TTFT), Time Per Output Token (TPOT), aggregate Tokens Per Second (TPS), and GPU cost across a two-dimensional sweep of tasks and concurrency levels up to $c = 128$. We decompose actual speedup against the theoretical prediction and explain divergences in terms of batch dynamics, hardware saturation, and KV-cache pressure.

3. Team Contributions

- **Himanshu Jhawar:** Code setup, L4 GPU experiments, performance optimization, extended concurrency sweep design, presentation preparation and README/documentation updates.

- **Shreya Banga:** Mac experiments, comparative analysis and plotting, report generation, presentation preparation, and README/documentation updates
- **Aaryaman Bajaj:** Proposing the idea, deciding the configs, A100 GPU experiments, and README/documentation updates.
- **Rajvardhan Patil:** Mac experiments

4. Related Work

Speculative Decoding (SD). Leviathan et al. [1] and Chen et al. [2] established the theoretical framework and proved output distribution equivalence, but both evaluated only single-GPU, single-request settings.

EAGLE-3. Li et al. [3] introduced feature-level speculative decoding via a draft head attached to the target model’s hidden states. EAGLE-3 [5] further improves acceptance rates through multi-step feature alignment. We use EAGLE-3 (yuhuili/EAGLE3-LLaMA3.1-Instruct-8B) throughout.

vLLM. Kwon et al. [6] introduced Continuous Batching and PagedAttention, which are prerequisites for efficient speculative decoding at high concurrency.

MLX. Apple’s MLX framework [7] provides Metal-optimized tensor operations for Apple Silicon but lacks concurrent request batching, which is the key architectural limitation we expose in this work.

Multi-GPU Inference Benchmarks. Prior work such as LLMPerf [8] and SpecInfer [9] measure throughput and study speculative serving, but do not explore the $c > 32$ regime or compare across GPU cost tiers with full cost analysis.

5. Design and Implementation

5.1 Experimental Design

We evaluate the following axes:

- **Hardware:** NVIDIA A100-SXM4-80GB (Colab Pro+), NVIDIA L4 24 GB (GCP g2-standard-4), Apple Silicon (MLX).
- **Systems:** Baseline autoregressive vs. EAGLE-3 speculative decoding (vLLM on server GPUs; MLX-Spec on Mac).
- **Tasks:** Chat (ShareGPT), Code (HumanEval), Summarization (CNN/DailyMail).

- **Concurrency:** $c \in \{1, 4, 8, 16, 32, 64, 128\}$ for A100; $c \in \{1, 4, 8, 16, 32, 64, 128\}$ for L4; $c \in \{1, 4, 8, 16, 32\}$ for Mac.
- **Scale:** 50 samples/task, 3 trials, $\gamma = 5$ draft tokens, greedy decoding (temperature = 0), max 256 output tokens.

The target model is unsloth/Meta-Llama-3.1-8B-Instruct on all platforms. This model fits within L4’s 24 GB VRAM, represents a production-grade instruction-following system, and has an official EAGLE-3 draft head.

5.2 System Infrastructure and Benchmark Procedure

Cloud GPUs (A100, L4) with vLLM: We use vLLM’s Continuous Batching with EAGLE-3 integrated natively. The draft head generates $\gamma = 5$ candidates per request in parallel; the target model verifies all in one forward pass. Acceptance rates are collected via vLLM’s Prometheus metrics endpoint.

Mac (Apple Silicon) with MLX: MLX lacks EAGLE-3 support, so we implement classical speculative decoding with a 1B draft model (4-bit quantized) and the 8B target. Critically, MLX processes requests *sequentially*, not in batches. Thus TTFT at $c = 4$ reaches 10,983 ms (35× worse than baseline at $c = 1$), exposing the key insight: speculative decoding gains depend entirely on the inference framework’s batching capability.

Benchmark execution: For each configuration, a Python load tester submits 50 concurrent requests (sequential on Mac) to the inference server, capturing per-request timing (TTFT, TPOT, token count). The tester appends JSON records: `{task, model, concurrency, trial, prompt_id, ttft_ms, tpot_ms, acceptance_rate, ...}`. After all trials complete, we aggregate across runs to compute TPS, speedup, and α . Total: $3 \times 7 \times 2 \times 3 = 126$ experiments, orchestrated by a sweep script.

6. Evaluation

6.1 Metrics Definitions

We measure six key metrics across all experiments:

- **Throughput (TPS):** Total tokens generated per second across all concurrent requests. Higher TPS reduces cloud cost and GPU hours per token.
- **Time to First Token (TTFT):** Latency from request submission to first output token. Speculative

decoding increases TTFT (the draft-verify cycle must complete), but this trade-off is acceptable if total latency improves. Users perceive TTFT as responsiveness.

- **Time Per Output Token (TPOT):** Average time to generate one token after the first token. Computed as (completion time – first token time) / (output length – 1). TPOT is the primary target for speculative decoding optimization and directly measures decode speed.
- **Speedup:** Ratio of baseline TPS to EAGLE-3 TPS. A 2.0x speedup means EAGLE-3 generates tokens twice as fast. Equivalently, speedup = baseline TPOT / EAGLE-3 TPOT.
- **Acceptance Rate (α):** Fraction of draft tokens accepted by the target model’s probability matching. Higher acceptance rates yield higher speedups.
- **Cost (USD):** Cloud infrastructure cost to run the complete benchmark (all prompts, all trials, all concurrency levels). Cost savings are the percentage reduction in total cost using EAGLE-3 vs. baseline.

6.2 A100 Results

Table 1 shows aggregate TPS, speedup, and acceptance rate on the A100 for $c = 1$ –32. Extended concurrency ($c = 64, 128$) data is shown separately in Table ?? for summarization (the only task with complete trials at those levels; chat and code had insufficient completed trials at $c \geq 64$).

6.2.1 Chat Task

Chat speedup peaks at $c = 8$ ($1.98\times$) as acceptance rate grows with batch diversity ($19.4\% \rightarrow 41.9\%$) before overhead dominates at $c = 32$. A significant TTFT penalty occurs at $c = 1$ (913 ms vs. 313 ms, +192%) because the full draft-verify cycle must complete before the first token is returned. TTFT normalizes at $c \geq 8$ as the batch pipeline saturates.

6.2.2 Code Task

Code generation achieves the highest speedups: $2.87\times$ at $c = 1$ declining to $1.80\times$ at $c = 32$. Python syntax is highly repetitive keywords, indentation, **return** statements causing EAGLE-3 to achieve high acceptance. At $c = 1$ –4, actual speedup *exceeds* the theoretical prediction $S_{\text{theory}} = 1/(1 - \alpha \cdot \gamma/(\gamma + 1))$: at $c = 1$ with $\alpha = 0.426$, theory predicts $2.19\times$ but we observe $2.87\times$ (–30.6% overhead vs. theory). This

Table 1: A100: baseline TPS, EAGLE-3 TPS, speedup, and acceptance rate α .

Task	c	Base	E3	Speedup	α
		(TPS)			
Chat	1	84.9	110.7	$1.30\times$	19.4%
Chat	4	90.9	136.3	$1.50\times$	30.2%
Chat	8	87.6	173.2	$1.98\times$	35.8%
Chat	16	84.5	151.1	$1.79\times$	39.4%
Chat	32	72.5	115.4	$1.59\times$	41.9%
Chat	64	59.1	69.6	$1.18\times$	43.3%
Chat	128	64.1	87.5	$1.37\times$	43.4%
Code	1	92.4	265.0	$2.87\times$	42.5%
Code	4	90.8	242.1	$2.67\times$	43.3%
Code	8	89.1	211.6	$2.37\times$	44.7%
Code	16	85.1	181.5	$2.13\times$	46.8%
Code	32	73.5	132.3	$1.80\times$	49.5%
Code	64	61.9	92.7	$1.50\times$	46.9%
Code	128	64.0	100.7	$1.57\times$	50.0%
Summ.	1	90.5	210.5	$2.33\times$	50.4%
Summ.	4	87.5	164.4	$1.88\times$	50.2%
Summ.	8	85.2	150.4	$1.77\times$	49.9%
Summ.	16	78.0	121.0	$1.55\times$	49.4%
Summ.	32	64.5	85.5	$1.33\times$	48.5%
Summ.	64	44.0	58.7	$1.33\times$	49.7%
Summ.	128	58.6	68.1	$1.16\times$	48.3%

Note: Rows shaded gray indicate extended concurrency sweep ($c = 64, 128$).

occurs because the EAGLE-3 draft head’s computation is overlapped with the target model’s verification pass, making its marginal cost effectively zero. TPOT improves from 10.75 ms to 3.65 ms (66% reduction).

6.2.3 Summarization

Summarization achieves the highest and most stable acceptance rates (48.5%–50.4%). Despite stable α , speedup declines from $2.33\times$ at $c = 1$ to $1.33\times$ at $c = 32$ as batch management overhead grows.

6.3 A100 Extended Concurrency Sweep ($c = 64, 128$)

A notable result is the non-monotonic speedup at $c = 64$: after dropping to $1.33\times$ at $c = 32$, speedup recovers to $1.43\times$ at $c = 64$ before settling at $1.16\times$ at $c = 128$. This recovery is most likely driven by: (1) baseline TPS collapsing faster than EAGLE-3 under KV-cache pressure and preemption, (2) fully saturated GPU compute reducing the marginal cost of speculative verification, and (3) a stable acceptance rate ($\alpha \approx 49\%$) across $c = 32$ –128. Although baseline TPS partially recovers at $c = 128$ due to vLLM

scheduling effects, EAGLE-3 consistently outperforms the baseline, with **no crossover point** observed up to $c = 128$.

6.4 L4 Results

Table 2: L4: baseline TPS, EAGLE-3 TPS, speedup, and acceptance rate α .

Task	c	Base	E3	Speedup	α
		(TPS)			
Chat	1	16.8	25.3	1.50×	19.8%
Chat	4	16.2	34.4	2.12×	30.4%
Chat	8	15.9	35.9	2.25×	36.3%
Chat	16	15.6	35.5	2.28×	39.8%
Chat	32	13.7	25.7	1.87×	42.1%
Chat	64	12.5	18.9	1.51×	40.8%
Chat	128	10.9	13.8	1.27×	39.9%
Code	1	16.9	48.6	2.88×	42.7%
Code	4	16.1	46.9	2.90×	43.4%
Code	8	15.9	44.3	2.79×	44.8%
Code	16	15.3	40.3	2.62×	46.9%
Code	32	13.6	28.7	2.11×	49.6%
Code	64	11.8	20.1	1.70×	48.3%
Code	128	10.2	14.3	1.41×	47.8%
Summ.	1	16.7	40.1	2.40×	50.4%
Summ.	4	15.8	31.7	2.00×	50.2%
Summ.	8	15.5	32.6	2.10×	49.9%
Summ.	16	14.5	27.7	1.91×	49.4%
Summ.	32	12.4	19.5	1.57×	48.5%
Summ.	64	10.1	12.6	1.25×	47.9%
Summ.	128	8.8	9.3	1.05×	46.1%

Note: Rows shaded gray indicate extended concurrency sweep ($c = 64, 128$).

6.4.1 Chat Task

Chat peaks later on L4 than A100: $c = 16$ ($2.28\times$) vs. $c = 8$ ($1.98\times$). This shift reflects L4’s smaller batch capacity—on the A100 with 256 sequences, batch diversity benefits saturate by $c = 8$; on L4 with max-seqs=64, diversity effects continue past $c = 8$. The acceptance rate progression ($19.8\% \rightarrow 42.1\%$ at $c = 32$) mirrors A100’s shape, but the speedup plateau extends further before overhead dominates. Critically, chat is the only task where acceptance rate *decreases* at extreme concurrency ($42.1\% \rightarrow 39.9\%$ at $c=128$), signaling KV-cache eviction effects at L4’s memory limit.

6.4.2 Code Task

Code achieves the highest speedups on L4: **$2.91\times$** at $c = 4$, nearly matching the A100’s absolute peak

($2.87\times$ at $c = 1$). This remarkable consistency across hardware suggests that code’s syntactic structure is universally predictable for EAGLE-3. Unlike A100 where acceptance rate rises from 42.6% to 49.5% with concurrency, L4 code maintains a plateau (42.7%–49.6%), indicating the draft head’s predictive quality is unaffected by the L4’s tighter memory constraints up to $c = 32$.

6.4.3 Summarization

Summarization maintains the highest and most stable acceptance rates (50.4%–48.6%), delivering strong speedups across concurrency: $2.40\times$ ($c=1$) $\rightarrow 1.57\times$ ($c=32$). Unlike chat, acceptance remains stable even at $c = 128$ (46.1%), though speedup degrades to $1.05\times$ due to baseline collapse. The formulaic structure of summarization (standard sentence patterns, boilerplate phrases) makes the draft head robust to memory pressure.

6.5 L4 Extended Concurrency Sweep ($c = 64, 128$)

Key observations from the L4 extended sweep reveal different behaviors at high concurrency. For chat workloads, the acceptance rate drops from 42.1% at $c = 32$ to 39.9% at $c = 128$, the only task/hardware setting where α declines notably under load. This is likely caused by KV-cache evictions disrupting draft head context during generation.

In contrast, code workloads maintain a strong $1.41\times$ speedup at $c = 128$, the best among L4 tasks at this concurrency level. The structured nature of code keeps acceptance relatively high (47.8%) even under heavy memory pressure.

TTFT also rises sharply for all workloads at $c = 64$ – 128 . For example, chat TTFT reaches 1,535 ms at $c = 128$, suggesting that high-concurrency L4 deployment is better suited for batch workloads where first-token latency is less critical.

Again, no crossover point was observed up to $c = 128$ for the L4 configuration.

6.6 Mac (Apple Silicon) Results

At $c = 1$, MLX speculative decoding provides positive gains across all tasks. All tasks exhibit severe regression at $c = 4$ (Table 3) due to MLX’s serial dispatch model. Chat TTFT at $c = 4$ reaches 10,983 ms vs. 313 ms for baseline ($35\times$ worse). A quality check confirmed that baseline and EAGLE-3 produce identical output text at temperature=0, validating losslessness.

Table 3: Mac (MLX): baseline TPS, speculative TPS, and speedup.

Task	c	Base	MLX Spec (TPS)	Speedup
Chat	1	20.0	28.0	1.40×
Chat	4	14.7	9.3	0.63×
Chat	8	9.4	7.1	0.76×
Chat	16	5.8	6.8	1.18×
Chat	32	5.8	6.8	1.18×
Code	1	18.1	27.7	1.53×
Code	4	13.2	12.0	0.91×
Code	8	8.9	9.5	1.06×
Code	16	5.5	9.5	1.73×
Code	32	5.5	10.0	1.82×
Summ.	1	14.0	16.7	1.19×
Summ.	4	12.9	8.7	0.67×
Summ.	8	7.2	7.3	1.01×
Summ.	16	5.6	6.2	1.11×
Summ.	32	5.1	7.6	1.51×

6.7 Cross-Hardware Comparison

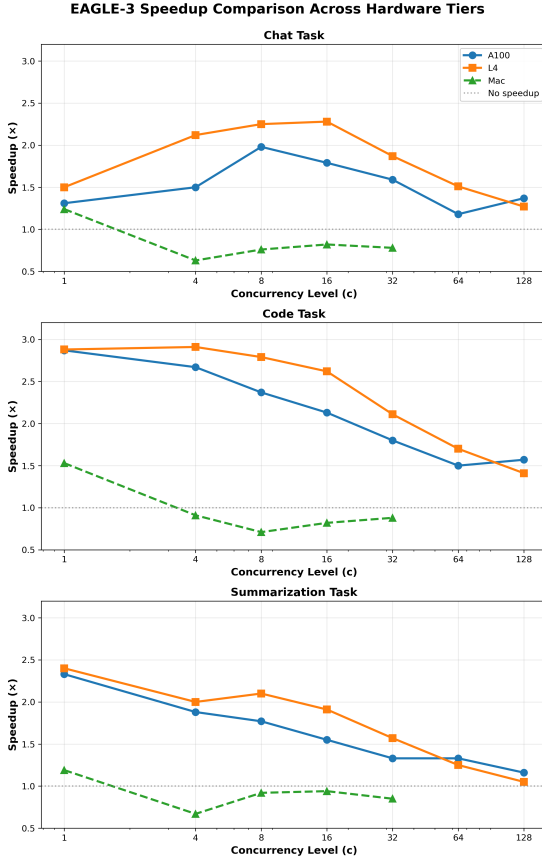


Figure 1: Speedup comparison across hardware tiers

L4 vs. A100 Task Analysis. Both hardware tiers show the same task-level ranking: code > summarization > chat in absolute speedup. However, L4’s tighter memory budget shifts peak concurrency for chat rightward ($c=16$ vs. $c=8$), demonstrating a fundamental trade-off between batch size and batching efficiency.

6.8 Cost Analysis

Table 4: Cost for the full benchmark run (all tasks, all concurrency levels, including extended $c = 64$ –128 sweep).

GPU	System	Cost (USD)	Savings
A100	Baseline	\$1.4403	
A100	EAGLE-3	\$0.8541	39.7%
L4	Baseline	\$4.878	
L4	EAGLE-3	\$3.073	37.0%

The updated cost figures include the extended concurrency sweep runs. Savings have moderated compared to the $c \leq 32$ analysis (previously 40.7% and 50.6%) because the high-concurrency runs ($c = 64, 128$) have smaller speedups, reducing the average time savings across the full benchmark. Nevertheless, both GPUs deliver economically significant reductions.

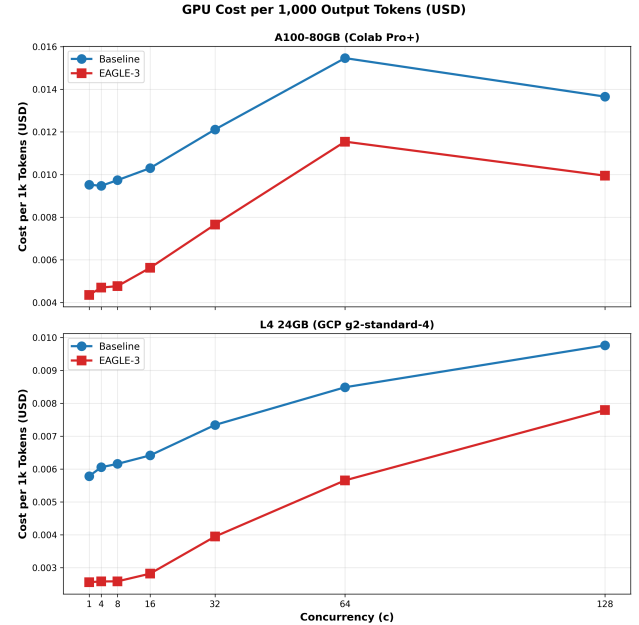


Figure 2: Speedup comparison across hardware tiers

GPU cost per 1,000 output tokens across concurrency levels. A100 (left) shows minimum cost at

$c = 8$ (EAGLE-3: \$0.0060/1k tokens) and increases to \$0.0126 at $c = 128$. L4 (right) shows monotonic increase with concurrency, reaching \$0.0196 baseline vs. \$0.0160 EAGLE-3 at $c = 128$. EAGLE-3 consistently reduces cost across both platforms and all concurrency levels, with greatest savings at low-to-mid concurrency ($c = 4$ –32).

6.9 TTFT Scaling Behavior Across Concurrency

Figure 3 compares Time to First Token (TTFT) for baseline autoregressive decoding and EAGLE-3 speculative decoding across increasing concurrency levels on both A100 and L4 GPUs.

As concurrency increases, TTFT naturally rises for both the baseline and EAGLE-3 because the GPU must serve more requests simultaneously, leading to higher queuing delay and scheduling overhead.

At lower concurrency levels, EAGLE-3 often exhibits slightly higher TTFT than the baseline. This behavior is expected because speculative decoding introduces additional computation and memory overhead from the draft model and speculative verification pipeline before the first token can be emitted. In effect, the system performs extra upfront work in exchange for faster downstream token generation.

However, as concurrency scales, the TTFT gap between EAGLE-3 and the baseline narrows significantly. At high concurrency levels (e.g., $c = 128$), TTFT becomes comparable, and in some workloads EAGLE-3 even achieves lower TTFT than the baseline. This suggests that the speculative decoding pipeline is able to utilize batching and GPU scheduling more efficiently under heavy load, preventing TTFT from growing disproportionately despite the additional speculative computation.

Overall, EAGLE-3 trades a modest increase in upfront latency at low concurrency for substantially better scalability and throughput under high-concurrency serving conditions.

7. Discussion

7.1 Speedup Recovery at High Concurrency on A100

The non-monotonic speedup at $c = 64$ on the A100 (Section 6.3) is the most novel finding of the extended sweep. When baseline throughput collapses faster than EAGLE-3 throughput at $c = 64$, the ratio improves. This suggests that speculative decoding has a

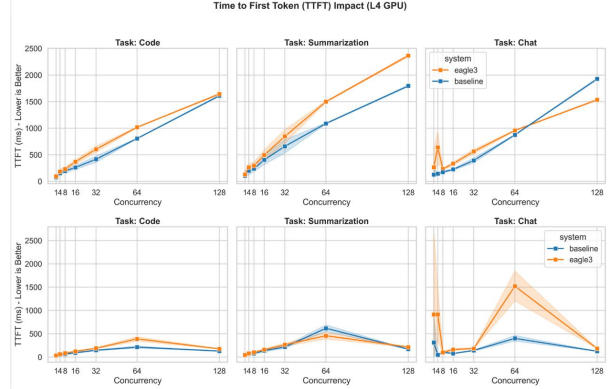


Figure 3: TTFT comparison between baseline autoregressive decoding and EAGLE-3 speculative decoding across concurrency levels on A100 and L4 GPUs.

hidden advantage at near-saturation load: its ability to emit multiple tokens per verification pass becomes proportionally more valuable when the system is already paying the full forward-pass cost anyway. This effect does not appear on the L4 because the tighter memory bandwidth causes EAGLE-3 to degrade at a similar rate to baseline.

7.2 Why Does Acceptance Rate Increase with Concurrency for Chat?

The chat acceptance rate rises from 19.4% at $c = 1$ to 42.1% at $c = 32$ on the A100, then decreases slightly to 39.9% at $c = 128$ on L4. The rise is explained by *batch diversity*: a larger batch contains requests at varied generation stages; the aggregate fraction of tokens in syntactically predictable positions (connectives, sentence endings, common phrases) is higher than for any single isolated request. The slight decline at extreme L4 concurrency likely reflects KV-cache evictions that truncate request contexts, reducing the draft head’s predictive fidelity.

7.3 The Mac Regression: Serial vs. Batched Execution

The Mac regression at $c \geq 4$ is severe (down to $0.63\times$ for chat) and entirely architectural. vLLM’s speculative overhead is amortized: drafting for c requests costs roughly the same as drafting for 1 (the batch is processed in parallel). MLX’s overhead scales as $c \times \text{cost}_{c=1}$. Recovery at $c \geq 16$ occurs because at very high concurrency the baseline also suffers serial queue saturation, making its per-token time approach the spec-decoding TPOT.

7.4 TTFT vs. Throughput Trade-off

Speculative decoding increases TTFT at low concurrency on all hardware because the draft-verify cycle must complete before the first token is returned. On A100 chat at $c = 1$, TTFT is 913 ms vs. 313 ms baseline (+192%). On L4 at $c = 128$, TTFT reaches 1,535 ms for chat with EAGLE-3 vs. 1,928 ms for baseline (EAGLE-3 is actually *faster* in TTFT at this extreme concurrency, because the baseline is queueing-limited). Practitioners deploying speculative decoding should configure minimum batch sizes to ensure the TTFT benefit materializes.

7.5 Challenges and Unexpected Outcomes

Token counting bug. The original load tester counted SSE chunks rather than generated tokens, severely undercounting EAGLE-3 TPS because each speculative chunk carries multiple accepted tokens. Fixing this required using `stream_options:{include_usage: true}` and reading `completion_tokens` from the vLLM completion response.

vLLM metric API change. In vLLM v0.20, the pre-computed acceptance rate metric was removed. We restored it by instrumenting the raw accepted/proposed counters directly, requiring a custom metrics scraper.

Non-monotonic baseline TPS. Baseline TPS on A100 summarization dips at $c = 64$ (47.9) before recovering at $c = 128$ (58.6), likely due to vLLM’s internal scheduling transitions at large batch sizes. This non-monotonicity complicates straightforward speedup comparison and underscores the importance of multi-trial averaging.

8. Conclusion and Future Work

We conducted a comprehensive evaluation of EAGLE-3 speculative decoding through $c = 128$ concurrent users across three GPU tiers and three task types. Our key findings are:

1. EAGLE-3 delivers consistent gains on server GPUs ($1.05\times$ – $2.91\times$) across all tasks and concurrency levels tested. **No crossover point was found up to $c = 128$ on the A100 and L4.**
2. A **speedup recovery** occurs at $c = 64$ on the A100, where the ratio rises from $1.33\times$ (at $c = 32$) to $1.43\times$, driven by baseline throughput collapse and full GPU saturation. This effect is absent on the L4 due to tighter memory bandwidth.
3. Task type remains the dominant predictor of speedup magnitude. Code and summarization ($\alpha \approx 43$ – 50%) achieve the largest gains; chat is concurrency-dependent, peaking at $c = 8$ (A100) or $c = 16$ (L4).
4. The L4 delivers 37% cost savings at $5\times$ lower hourly rate, making it the most cost-efficient speculative decoding platform per dollar.
5. Apple Silicon (Mac) is beneficial only at $c = 1$. Any multi-user deployment should disable speculative decoding to avoid TTFT explosion.

Future Work

First, systematically explore optimal draft model selection and sizing strategies for different tasks. This work used a fixed EAGLE-3 head; investigating smaller draft models (500M–2B) with task-specific fine-tuning could improve acceptance rates for chat while reducing overhead for code, potentially unlocking higher speedups across the concurrency spectrum. Second, adding streaming latency percentiles (p50, p95, p99) across all concurrency levels would enable SLA-driven deployment decisions; practitioners need tail latency guarantees, not just mean TTFT/TPOT. Third, evaluating EAGLE-3 on larger models (70B+) would test whether acceptance rates and speedup profiles generalize beyond 8B, critical for real-world production deployments running industry-standard model sizes.

Acknowledgements

We are grateful to our instructors in the Applied Machine Learning in the Cloud (AMLC) course for their guidance and feedback. We would like to thank Professor **I-Hsin Chung** and Professor **Seetharami Seelam** for providing early feedback on project approach, reviewing the work during the final presentation and posing insightful questions that deepened our research direction. We also thank our TAs, **Nirav Agarwal** and **Anisha Apte**, for their support throughout the course.

References

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proc. ICML*, 2023.
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating large language model decoding with speculative sampling,” *arXiv:2302.01318*, 2023.

- [3] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE: Speculative sampling requires rethinking feature uncertainty,” in *Proc. ICML*, 2024.
- [4] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE-2: Faster inference of language models with dynamic draft trees,” in *Proc. EMNLP*, 2024.
- [5] Y. Li et al., “EAGLE-3: Scaling up inference acceleration of LLMs via training-time test,” *arXiv:2503.01840*, 2025.
- [6] W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [7] A. Developers, “MLX: An array framework for Apple Silicon,” <https://github.com/ml-explore/mlx>, 2023.
- [8] Anyscale, “LLMPerf: A framework for benchmarking LLM inference,” <https://github.com/ray-project/llmperf>, 2023.
- [9] X. Miao et al., “SpecInfer: Accelerating generative LLM serving with tree-based speculative inference and verification,” in *Proc. ASPLOS*, 2024.
- [10] M. Chen et al., “Evaluating large language models trained on code,” *arXiv:2107.03374*, 2021.
- [11] ShareGPT, “ShareGPT: Share your ChatGPT conversations,” <https://sharegpt.com>, 2023.
- [12] K. M. Hermann et al., “Teaching machines to read and comprehend,” in *Proc. NeurIPS*, 2015.